Übungsblatt 8: Besitz, Fortschritt und Fehlerreaktion

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Ziele und Ablauf

Sie erklären Konkurrenzfehler, Warteprotokolle, Watchdogfenster und Fehlerreaktionen. Grundlage sind Vorlesungen 1–8. HSEM bedeutet Hardware Semaphore, IWDG Independent Watchdog, WWDG Window Watchdog, EWI Early Wakeup Interrupt und MPU Memory Protection Unit.

Verbindlicher Übungskern ist die Klausurvorbereitung auf Papier. Die angegebenen Punkte dienen zur Selbsteinschätzung; die Modulnote entsteht durch die Papierklausur. Alle Rechen- und Analyseaufgaben sind mit den angegebenen Daten ohne Hardware oder erfolgreiche frühere Programmieraufgaben lösbar. Begründen Sie Entscheidungen und nennen Sie Modellannahmen.

Abschnitt	Minuten	Zweck
Papierkern mit Besprechung	50	32 Orientierungspunkte
Transfer und Besprechung	10	Rechnen und begründen
Praxisfall: vorbereitete Demonstration oder Papieralternative	15	Vorhersagen, erklären, Grenzen nennen

Der Übungsblock umfasst **75 Minuten**. Die Praxisphase umfasst etwa fünf Minuten Vorhersage, höchstens fünf Minuten Demonstration und fünf Minuten Erklärung. Ohne funktionierenden Aufbau bearbeiten Sie in denselben 15 Minuten den vollständig vorgegebenen Papierfall. **Nach spätestens drei Minuten technischer Fehlersuche wird zum Papierfall gewechselt.** Dadurch bleibt der Übungsblock auch bei Setup-Problemen nutzbar.

Für eine Demonstration verwendet die Lehrperson **STM32CubeMX**, **VS Code mit STM32 VS Code Extension** und **NUCLEO-H755ZI-Q / STM32H755ZIT6**. Sie bereitet die geprüften Projektvarianten vor dem Termin vor. Installation, Projektgenerierung, Integration, Regenerierungsprüfung, Build und Flash sind keine Pflichtaufgaben im Übungstermin. Eine eigene Implementierung ist freiwillige Vertiefung. Alle Zahlen dieses Blatts sind Lehrannahmen, soweit sie nicht ausdrücklich als eigene Messwerte gekennzeichnet sind.

Der Übungsblock ergänzt 90 Minuten Erklärung und 15 Minuten Quiz zum Termin mit 180 Minuten.

Aufgabe 1 · Konkurrenz und Endwert · 8 Punkte · 12 Minuten

Zwei Kerne sollen einen gemeinsamen Zähler von fünf aus jeweils einmal erhöhen. Einzelne ausgerichtete Wortzugriffe gelten im Modell als ungeteilt. Lesen, Addieren und Schreiben sind getrennte Schritte.

1. Geben Sie eine Reihenfolge mit Endwert sechs an. Nennen Sie die privaten Kopien und den gemeinsamen Speicher. Begründen Sie, warum `volatile` und ein Schloss nur um das Schreiben nicht ausreichen. **(3 P)**
2. Ein neuer endlicher Lauf startet bei null. Je Kern werden 100 Erwerbsversuche ausgeführt. M7 meldet 97 und M4 95 erfolgreiche Inkremente. Beide Abschlussmeldungen sind sichtbar, der Speichervertrag gilt und es gibt keinen Überlauf. Was ist der Sollwert? Beurteilen Sie auch einen ungeschützten Lauf, der zufällig diesen Wert erreicht. **(2 P)**
3. Zwei M4-Tasks benutzen dieselbe HSEM-Nummer und beide FastTake. Warum reicht das nicht? Was bedeutet ein zweites HAL_OK desselben Besitzers, und wie wirken eine einzelne Freigabe und ein zusätzliches lokales Mutex? **(3 P)**

Ihre Lösung

(Notieren Sie Ihre Schritte und Begründungen hier.)

Musterlösung Aufgabe 1

1. M7 liest fünf, M4 liest fünf, beide erhöhen privat auf sechs, M7 schreibt sechs, M4 schreibt sechs. `volatile` erzeugt keinen gegenseitigen Ausschluss. Ein Schloss nur um das Schreiben lässt die alte Registerkopie bestehen. Richtig geschützt werden Lesen, Ändern und Schreiben gemeinsam. Pro nachvollziehbarer Reihenfolge und den beiden Schutzbegründungen je ein Punkt.
2. $97 + 95 = 192$. Die Versuchszahl 200 ist keine Erfolgssumme. Ein ungeschützter Treffer 192 ist mit einer günstigen Reihenfolge vereinbar und beweist keine Freiheit von Konkurrenzfehlern. Je ein Punkt.
3. FastTake verwendet ProcessID null. Auf demselben Kern erscheinen die beiden Tasks daher als dasselbe Besitzpaar. Die HAL-Rückleseprüfung kann denselben Besitz erneut bestätigen, ohne einen Rekursionszähler zu erhöhen. Ein Release gibt ihn frei. Ein lokaler Mutex serialisiert diese Tasks vor dem HSEM-Erwerb; die globale Sperrordnung muss eingehalten werden. Je ein Punkt für Kennung, Wiedererwerb/Freigabe und lokale Serialisierung.

Aufgabe 2 · Aufwecken, Frist und Sperrordnung · 8 Punkte · 12 Minuten

Ein M4-Task scheitert beim Take. M7 gibt frei, bevor M4 seine Notification aktiviert. Erst danach ruft M4 Wait auf. Modell: genau ein wartender Task, ein gespeichertes Thread-Flag und keine weitere garantierte Freigabe.

1. Warum kann der Task unnötig blockiert bleiben? **(1 P)**
2. Geben Sie die richtige Reihenfolge aus Scharfschalten, Take, Flagbehandlung, Wait und erneutem Take an. Die Referenz-HAL deaktiviert ausgelöste Notification-Bits im Handler. Berücksichtigen Sie das und nennen Sie die Rolle der zulässigen Interruptpriorität. **(3 P)**
3. Der gesamte Versuch beginnt bei 0 ms und hat 20 ms Frist. Meldungen kommen bei 8 und 17 ms. Ein erneuter Take scheitert bei 9 ms. Welche Restzeiten gelten? Warum wäre jeweils ein neues 20-ms-Warten falsch? **(2 P)**

4. M7 hält Schloss 1 und will 2, M4 hält 2 und will 1. Nennen Sie eine vorbeugende Entwurfsregel und erklären Sie, warum ein Timeout allein die bereits gehaltenen Schlösser nicht löst. **(2 P)**

Ihre Lösung

(Skizzieren Sie das Protokoll und die Restzeiten hier.)

Musterlösung Aufgabe 2

1. Die Freigabe erfolgt vor der Meldebereitschaft und wird nicht für Wait gespeichert. Ohne weitere Freigabe bleibt ein Lost Wakeup, verlorenes Aufwecken. Ein Punkt.
2. Gültiges Handle und alte eigene Ereignisse klären, Notification aktivieren, Take versuchen. Bei Erfolg Meldung deaktivieren, kurz arbeiten und passend freigeben. Bei Fehlschlag das gespeicherte Flag **nicht** zwischen Take und Wait löschen. Wait benutzt die Restzeit. Nach dem Aufwecken neu scharfschalten und erneut Take versuchen. Der Callback setzt nur das Ereignis mit ISR-zulässiger Schnittstelle und zulässiger Priorität, keinen blockierenden Mutex. Die Meldung ist kein Besitz. Je ein Punkt für Reihenfolge, Flag/Neuerwerb und Rearming/Priorität.
3. Bei 8 ms bleiben 12, bei 9 bleiben 11 und bei 17 bleiben 3 ms. Neue volle Intervalle würden die vereinbarte Gesamtfrist verlängern. Je ein Punkt für Zahlen und Begründung.
4. Alle Beteiligten nehmen Schlösser in global gleicher Reihenfolge, beispielsweise 1 vor 2, einschließlich lokaler Mutexes. Der Timeout beendet nur das Warten; eigene gehaltene Schlösser müssen im definierten Abbruchpfad freigegeben und Teilschritte behandelt werden. Je ein Punkt.

Aufgabe 3 · Zwei Watchdog-Rechnungen · 8 Punkte · 14 Minuten

Für den WWDG2 gelten hier $f_{\mathrm{PCLK}} = 100\,\mathrm{MHz}$, Vorteiler $P = 8$, Nachladewert $R = 127$ und Fenster $W = 80$. Erlaubt ist $63 < T \leq 80$. EWI wird bei $T = 64$ angefordert, Reset beim Schritt nach 63. **Idealphase:** erster Abwärtsschritt einen vollen Tick nach dem Start.

1. Berechnen Sie mit $\Delta t = 4096P/f_{\mathrm{PCLK}}$ den Tick und die idealen Zeiten bis Fensteröffnung, EWI und Reset. Ist Gleichheit $T = W$ erlaubt? **(3 P)**
2. Bewerten Sie Refresh nach 5, 18 und 25 ms. Warum ist `osDelay(18)` ohne weitere Bedingungen kein Nachweis von 18 ms tatsächlicher Wartezeit? **(1 P)**
3. Beim IWDG ist das Fenster ausgeschaltet. $R = 500$, $P = 32$. Nehmen Sie rein didaktisch $f_{\mathrm{LSI,min}} = 28\,\mathrm{kHz}$ und $f_{\mathrm{LSI,max}} = 36\,\mathrm{kHz}$ an. Berechnen Sie früheste und späteste Ablaufzeit mit $(R+1)P/f$. Welche davon ist für rechtzeitigen Refresh maßgeblich? **(2 P)**
4. Die EWI-Reaktion hat nominal einen WWDG-Tick. Angenommene Eintrittslatenz 40 µs, Handler-Grundarbeit 20 µs, Reserve 30 µs. Welches Arbeitsbudget bleibt? Nennen Sie eine Bedingung, unter der der Callback gar nicht rechtzeitig läuft. **(2 P)**

Ihre Lösung

(Zeigen Sie Einheiten, Rechenweg und Annahmen hier.)

Musterlösung Aufgabe 3

1. $4096 \cdot 8/10^8 = 0{,}00032768\,\text{s} = 0{,}32768\,\text{ms}$. Öffnung nach $127 - 80 = 47$ Schritten: 15,40096 ms. EWI nach 63 Schritten: 20,64384 ms. Reset nach 64 Schritten: 20,97152 ms. $T = W = 80$ ist erlaubt. Je ein Punkt für Tick, Grenzrechnung und Gleichheit. In echter Hardware kann die Vorteilerphase den ersten Schritt um bis zu etwa einen Tick verschieben.
2. 5 ms: zu früh, unmittelbarer Reset ohne nötigen EWI. 18 ms: nominal 54 Schritte, $T = 73$, im Fenster. 25 ms ohne Refresh: Reset bereits vorher. `osDelay` benutzt Kernelticks; Frequenz, Tickphase und spätere Einplanung des Tasks gehören in den Vertrag. Ein Punkt für vollständige Bewertung.
3. $501 \cdot 32 = 16032$. Frühestes Ende bei höchster Frequenz: $16032/36000 = 0{,}445333\,\text{s}$, gerundet 445,33 ms. Spätestes Ende: $16032/28000 = 0{,}572571\,\text{s}$, gerundet 572,57 ms. Rechtzeitiger Refresh richtet sich nach der frühesten Grenze mit Reserve. Die Werte sind **keine Datenblattgrenzen**. Je ein Punkt für korrektes Band und Interpretation.
4. $327{,}68 - 40 - 20 - 30 = 237{,}68\,\mu\text{s}$. Der EWI ist maskierbar; globale Interruptsperre, deaktivierter IRQ oder ausreichend lange Verdrängung durch dringlichere Handler können die Reaktion verhindern. Auch zu frühes Füttern kann direkt ohne EWI resetten. Je ein Punkt.

Aufgabe 4 · Fortschritt und Ausgangsschutz · 8 Punkte · 12 Minuten

Ein Supervisor hat bei jetzt 25 ms zuletzt neue Epochen von Worker A bei 20 ms und von B bei 10 ms gesehen. A hat eine Frischefrist von 10 ms, B von 30 ms. Alle Zeiten benutzen die lokale Supervisor-Uhr.

1. Sind beide Nachweise frisch? Was beweist eine weiterlaufende Idle-Task bei zwei blockierten Workern? **(2 P)**
2. Für die Reaktion ab tatsächlichem Stillstand gelten die angenommenen Höchstwerte: Frischefrist 30 ms, Abfrageverzug 5 ms, möglicher Meldeverzug 2 ms, Watchdogrest 21 ms, Startreaktion 8 ms. Reicht dies für eine Anforderung von 60 ms? **(2 P)**
3. Eine passende No-Access-Region ist auf M7 aktiv, ohne erlaubende Überlappung. Wird ein privilegierter gewöhnlicher Interrupt-Handler daran gehindert zu schreiben? Gilt dieselbe Region für DMA oder M4? Ein Arrayüberlauf landet innerhalb einer erlaubten Region: muss er faulten? **(2 P)**
4. Ein isoliertes aktives High-Signal ENABLE wurde im EWI auf Low gesetzt. Erklären Sie, warum zusätzlich der Pegel während Reset und Boot geregelt werden muss. Nennen Sie eine passende Lehrbeschaltung und eine Bootregel. **(2 P)**

Ihre Lösung

(Begründen Sie jede Schutzgrenze hier.)

Musterlösung Aufgabe 4

1. Alter A: $25 - 20 = 5\,\text{ms} \le 10\,\text{ms}$, B: $25 - 10 = 15\,\text{ms} \le 30\,\text{ms}$. Beide frisch. Idle beweist nur, dass ihr eigener lokaler Pfad Rechenzeit bekommt, keinen Workerfortschritt und keinen neuen periodischen Tick. Je ein Punkt.
2. $30 + 5 + 2 + 21 + 8 = 66\,\text{ms}$. Die Modellgrenze überschreitet 60 ms. Die Anforderung ist mit diesen Annahmen nicht erfüllt. Der Meldeverzug berücksichtigt eine bereits fertiggestellte Runde, die erst nach dem Stillstand beobachtet wird. Je ein Punkt für Summe und Entscheidung.

3. Ja, auch ein gewöhnlicher privilegierter Handler unterliegt der aktiven passenden No-Access-Region dieses Kerns. DMA und M4 werden von der M7-MPU nicht geprüft. Ein Zugriff außerhalb des C-Arrays, aber innerhalb erlaubter Adressen muss keinen MPU-Fault auslösen. Je ein Punkt für lokale Rechte und äußere Grenzen.
4. Beim Reset kann der Pin hochohmig werden, sodass ein vorheriger Low-Schreibwert keine Pegelzusage mehr ist. Ein externer Pull-down hält die isolierte aktive-High-Anzeige inaktiv. Der Startpfad initialisiert Low und gibt erst nach Prüfung wieder frei. EWI allein kann wegen Maskierung oder frühem Reset fehlen. Je ein Punkt für Resetzustand und Beschaltung/Startregel.

Transfer: Reset ist noch kein inaktiver Ausgang · 10 Minuten

Zusätzliche Klausurübung ohne weitere Punkte. Ein isolierter aktiver High-Ausgang ENABLE hat einen externen Pull-up. Beim Reset wird der GPIO hochohmig. Das Programm setzt ENABLE erst 8 ms später auf Low. Ein zu früher WWDG-Refresh löst einen Reset aus; für diesen Fall ist kein EWI garantiert.

1. Welcher Pegel ist während der acht Millisekunden zu erwarten? Erfüllt dies „auch während Reset und Boot inaktiv“?
2. Warum reicht der Satz „der EWI setzt Low“ nicht? Formulieren Sie eine passende Beschaltungs- und Bootregel für die isolierte Anzeige.
3. Würde ein regelmäßiger Refresh durch die Idle-Task zusätzlich den Fortschritt aller Worker nachweisen?

Musterlösung Transfer 8

Der Pull-up zieht den hochohmigen Ausgang auf High und damit aktiv; der Vertrag ist verletzt. Der Reset kann ohne EWI eintreten, und ein vorheriger Low-Schreibwert hält den hochohmigen Pin nicht Low.

Für die aktive-High-Lehranzeige ist ein passend dimensionierter externer Pull-down mit geprüften Pegeln geeignet. Der Startpfad initialisiert inaktiv und erlaubt die Freigabe erst nach den vereinbarten Prüfungen. Das ist ein Lehrentwurf, kein allgemeiner Maschinensicherheitsnachweis. Idle-Lauf beziehungsweise Refresh belegt nicht die frischen vollständigen Kontrollrunden der erforderlichen Worker.

Praxisfall · 15 Minuten

Praxisfall 8: Erfolgssumme und Endwert

Papieralternative, synthetische abgeschlossene Zählerläufe: Jeder Kern führt 100 Versuche aus. Die Kennungen, Sichtbarkeit und Abschlussbedingungen sind im Modell gültig, Diagnosezähler laufen nicht über. In G schützt ein gemeinsamer Besitzvertrag das ganze Inkrement. U hat denselben nicht cachebaren Speicher, aber keinen gegenseitigen Ausschluss.

Lauf	Erfolge M7	Erfolge M4	Endwert ab null
G, geschützt	98	99	197
U1, ungeschützt	100	100	198
U2, ungeschützt	100	100	200

1. Bestimmen Sie je Lauf Sollsumme und Differenz. Welche Fehlversuchszahlen gehören zu G?

2. Beweist U2 die Freiheit von Lost Updates? Darf eine passende Datenbilanz als Frist- oder Fortschrittsgarantie für alle Zustände ausgegeben werden?
3. Bei einem vierten Lauf fehlt M4s Abschlussmeldung. Darf dessen momentaner Endwert in derselben Weise abgenommen werden?

Bei vorbereitetem Board: Die Lehrperson wählt **einen** kurzen Vergleich: geschützter/ungeschützter Zähler oder die separat vorbereitete WWDG-Station. Die Gruppe erklärt die Beobachtung anhand der Papiermodelle. Watchdog, Shared-Layout und GPIO-Verdrahtung werden im Termin nicht neu eingerichtet.

Musterlösung Praxisfall 8

G: Sollsumme $98 + 99 = 197$, Differenz null; zwei M7- und ein M4-Fehlversuch. U1: Sollsumme 200, Endwert zwei darunter. U2: Sollsumme 200, passender Endwert. Die günstige U2-Reihenfolge widerlegt das mögliche Lost Update nicht. Nicht cachebar ersetzt keinen gegenseitigen Ausschluss.

Eine abgeschlossene Datenbilanz betrifft diesen Lauf, keine allgemeine Zeit- oder Fortschrittsgarantie. Fehlt ein Abschluss, liegt kein gültiger finaler Prüfpunkt vor. Ein momentaner Zähler wird nicht als abgeschlossenes Ergebnis bewertet. Alle Werte der Tabelle sind Lehrdaten und keine Boardmessungen.

Ergebnis und freiwillige Vertiefung

Halten Sie die begründeten Papierlösungen (32 Orientierungspunkte), Ihre Transferantworten und die Erklärung des Praxisfalls fest. Projektdateien, Versionslisten, Flashnachweise und Messprotokolle sind keine Pflichtabgabe dieses Übungsblocks. Für das Verständnis des nächsten Blatts ist keine erfolgreiche Firmwareintegration erforderlich.

Die freiwillige Programmiervertiefung enthält die bisherige technische Vorbereitung und Integration. Sie richtet sich an die Lehrperson sowie an Studierende, die außerhalb des Termins mehr Praxis wünschen. Ein fehlender Boardlauf beeinträchtigt die Bearbeitbarkeit der Papieraufgaben nicht. In eigenen Protokollen bleiben ungemessene Zeitfelder leer; beobachtete Stichprobenmaxima sind keine allgemeine Worst-Case-Garantie.

Quellen: Vorlesung 8; Giorgio C. Buttazzo, *Hard Real-Time Computing Systems*; die in der Vorlesung und der freiwilligen Integrationsanleitung genannten Herstellerdokumente.